1-Track elegido y justificación

- Se eligió el Track Backend / Integración.
- **Justificación**: Cuando se produce un problema con los vuelos en una línea aérea, lo complejo es lograr datos que sean consistentes, poder aplicar las políticas de la empresa y dado que esto tiene un impacto muy alto tanto para el pasajero como para la empresa, es mejor concentrarse en el backend para poder trabajar con los datos y limpiarlos, aplicar las reglas del sistema y los guardavías transaccionales que sean requeridos.

2. Instrucciones para revisar o ejecutar:

Se entrega el archivo latam-0.0.1-SNAPSHOT.jar que se debe compilar en un docker con SpringBoot / NGINX ya configurados, y que se recompila con la instrucción: docker compose up

3. Alcance efectivamente implementado (Vertical Slice)

- Se armó un orquestador en java DisruptionEngineService.java que resuelve de extremo a extremo el caso base que se entregó con 4 etapas
    - **1 - Ingesta y Resolución:**
        - saneado de las fuentes de datos -se descarta LEGACY_MIRROR.
        - Se verifica RECORDED_AT, lo que nos permite manejar la temporalidad, y se toma como canónico el OPS_CORE si tenemos que decidir.
    - **2 - Detección de Impacto:**
        - se verifican los timestamp de los segmentos S1 y S2, lo que nos permite detectar que S1 no deja tomar S2, por lo que se activa missed_connection=true.
    - **3 - Evaluación de políticas de la compañía:**
        - Inspecciona los pasajeros y permite detectar al menor.
    - **4 - Acciones de Resolución:**
        - Preselecciona opción válida 01 (vuelo sin costo con asientos disponibles)
        - Bloquea la auto emisión con create_rebooking_request, y aplica create_human_case en la cola DISRUPTION_L2 -acá hay un SLA de 45 minutos que hay que tener en cuenta-
        - Genera las notificaciones que correspondan al frontend.

4 - test y/o validaciones disponibles.

- Test 1: testDegradedSourceAndMissedConnection
    - Valida calidad de datos (aquí se deben filtrar contradicciones), como DEGRADED.
    - Calcula los horarios del itinerario
- Test 2: testGuardrailForMinorPassenger
    - Aquí se reducen los riesgos (un menor)
    - Se usa el principio del human_in_loop
- Test 3: testViableOptionsSelection:
    - Se encarga de revisar si las recomendaciones son inviables.
    - Valida que no se cobre al pasajero de forma indebida.

5 - Las limitaciones conocidas:

- Datos fijos -persistentes-, estamos usando los mocks que nos entrega el documento base.
- No se resuelven los cobros, no hay control de moneda, pagos, pasarelas de pago
- Transacciones inconsistente: uno de los métodos más importantes que es el create_rebooking_request, si falla, la transacción nunca termina -es un punto crítico-